Temporal Superposition in GPT-2: How Temporal Time is Encoded and Distributed Across Layers

Sam Brown Peyton Calvert

Your Institution

{sbrown, pcalvert}@yourinstitution.edu

Abstract

We investigate how GPT-2-small encodes historical time in its residual stream using sparse autoencoders (SAEs). TODO: fill in findings summary.

1 Introduction

Large language models have demonstrated their ability to "know" historical facts, but we have limited understanding of how that knowledge is stored internally. Understanding the geometric structure of factual knowledge in LLMs is a foundational question for mechanistic interpretability.

2 Related Work

TODO

3 Background

3.1 Residual Stream

Transformer models process tokens by passing representations through a sequence of attention and MLP layers. These layers write incremental updates into a shared vector called the residual stream. Formally if $\mathbf{x}^{(\ell)} \in \mathbb{R}^d$ denotes the residual stream at layer ℓ, each layer computes $\mathbf{x}^{(\ell+1)} = \mathbf{x}^{(\ell)} + f^{(\ell)}$, where $f^{(\ell)}$ is the combined attention and MLP transformation. This additive structure means the residual stream accumulates information across depth, and that respresentations at any layer can be read as a type of superposition of all contributions made so far. In this work we probe the residual stream of GPT-2 Small Radford et al. [2019], a 117 million parameter transformer with a hidden dimension of d = 768.

3.2 Superposition

A naive expectation is that each dimension of the residual stream encodes one interpretable feature. In practice, the number of concepts a model must represent exceeds its hidden dimension d ($\mathbb{R}^{768}$ in this paper). As a result, models exploit the fact that high dimensional spaces can accommodate many nearly-orthogonal directions. This phenomenon is called superposition and it allows a model to store a large amount of features at the same time. Features in superposition are distributed across many neurons.

3.3 Sparse Autoencoders

Sparse autoencoders (SAEs) offer a way to recover the underlying precise features from an ambiguous residual stream Cunningham et al. [2024]. An SAE consists of an encoder that projects the d-dimensional activation into a much large latent space $\mathbb{R}^{d_{\text{SAE}}}$ with $d_{\text{SAE}} \gg d$, followed by a ReLU that enforces sparsity and a linear decoder that reconstructs the original activation. Training minimises reconstruction error subject to an L_1 penalty on the latent activations, encouraging each input to be explained by only a small number of active features at a time. The resulting latent dimensions have been shown to correspond to interpretable, monosemantic concepts Cunningham et al. [2024], Bloom [2024]. In this work we use the SAE of Bloom [2024] trained on GPT-2 Small's residual stream.

4 Method

4.1 Model and SAE

We do all experiments on GPT-2 Small Radford et al. [2019], a decoder-only transformer with 12 layers, 12 attention heads, and a residual stream dimension of d = 768, totalling approximately 117 million paramters. GPT-2 Small is a well-studied model in the context of mechanistic interpretability, making it a good model for probing internal representations.

To decompose residual stream activations, we use the suite of SAEs released by Bloom [2024], trained on GPT-2 Small's residual stream under the release identifier `gpt2-small-res-jb`. Each SAE is trained on the `hook_resid_pre` activation at its corresponding layer – that is, the residual stream state *before* the attention and MLP transformations of that layer have been applied. The SAE architecture uses a linear encoder with ReLU activation followed by a linear decoder, with an expansion factor of 64×, giving a latent dimension of $d_{\text{SAE}} = 49{,}152$. For each layer ℓ we load the corresponding SAE independently, allowing us to probe how feature representations evolve across depth.

All activations are extracted at the final token position of each prompt, as this is the position at which the model has processed the full input and is most likely to have integrated contextual information into a coherent representation.

4.2 Prompt Design

TODO (May still try different prompts like prof mentioned... come back to)

4.3 Temporal Axis Measurement: Linear Concepts

To quantify how strongly a layer encodes temporal order for historical year prompts, we measure the alignment between the model's internal representations and chronological position. For each layer l, we pass the year prompt activations through the pretrained SAE encoder and reconstruct them via the decoder:

$$\hat{x}_i = \mathbf{W}_{\text{dec}}\mathbf{a}_i \in \mathbb{R}^{768}$$

where $\mathbf{a}_i \in \mathbb{R}^{49152}$ is the sparse feature activation vector for prompt i. This reconstruction discard variance not captured by the SAE, meaning our metric reflects temporal structure as expressed through the SAE's feature basis specifically.

We apply Principal Component Analysis (PCA) to the $(N \times 768)$ reconstruction matrix, retaining a single component (PC1), and compute the absolute Pearson correlation between PC1 scores and numeric year values:

$$|r| = \left|\text{corr}\left(\text{PC1}(\hat{\mathbf{X}}),\ \mathbf{y}\right)\right|$$

where $y \in \mathbb{R}^N$ contains the numeric year for each prompt, with pre-common-era dates represented as negative integers. We use PC1 because it captures the direction of maximum variance in the reconstruction. If the model encodes temporal position as a linear axis, that axis should dominate the variance structure and align with PC1. The absolute value discards PCA's arbitrary sign convention. We refer to this quantity as the baseline temporal correlation $|r|_{\text{base}}$ at a given layer, and use drops in $|r|$ under feature ablation as the primary signal in our superposition analysis. described in Section 4.5.

4.4 Temporal Axis Measurement: Cyclic Concepts

For cyclic concepts – days of the week, months of the year, and days of the month – a linear correlation metric is inappropriate. Day 7 and day 1 are adjacent in the cyclic ordering but greatly separated under any linear projection, meaning Pearson r will be near zero even for a geometrically well-structured representatiion. We therefore adopt a circular measurement pipeline.

We retain five PCA components from the SAE reconstruction and fit an algebraic circle to the PC2/PC3 plane using the Coope(1993) least squares method Coope [1993]. PC1 is deliberately excluded as it captures non-cyclic variance. PC1 captures whether the text refers to a weekday or month and the PC2/PC3 captures the circular structure. For each prompt we compute its angular position relative to the fitted circle center (c_x, c_y):

$$\theta_i = \arctan 2\left(\text{PC3}_i - c_y,\ \text{PC2}_i - c_x\right)$$

We then compute the Fisher circular-linear correlation between angular position and the integer label $\mathbf{y}$. A standard Pearson correlation is inappropriate here because θ_i is a circular variable — day 7 and day 1 are adjacent in the cyclic ordering but nearly π apart in angle, causing Pearson r to underestimate the true correspondence. The Fisher metric instead projects the angular variable onto $\cos\theta$ and $\sin\theta$ and measures how well both components jointly predict the label, correctly treating the concept as periodic.

$$r_{\text{angular}} = \sqrt{\frac{r_{xc}^2 + r_{xs}^2 - 2r_{xc}r_{xs}r_{cs}}{1 - r_{cs}^2}}$$

where $r_{xc} = \text{corr}(\cos\theta, \mathbf{y})$, $r_{xs} = \text{corr}(\sin\theta, \mathbf{y})$, and $r_{cs} = \text{corr}(\cos\theta, \sin\theta)$. This metric is bounded in $[0, 1]$ and correctly treats the label as periodic, with $r_{\text{angular}} = 1$ indicating that the label perfectly predicts angular position on the circle. For the ablation scan in Phase 5, drops in r_{angular} under single-feature ablation replace drops in $|r|$ as the signal metric, with all other aspects of the pipeline held constant.

4.5 Feature Clustering

Given the high dimensionality of the SAE feature space ($d_{\text{SAE}} = 49,152$), directly ablating all features is computationally unmanageable and statistically uninformative. We therefore apply a two stage filtering procedure to identify a coherent subset of features likley to be encoding temporal structure, which then forms the canidate set for the ablation scan.

We First filter to *active* features, those that fire on at least two prompts in the input set. For each active feature f_i we compute a one-way ANOVA F-score across epoch category groups, where each group contains the feature activations for all prompts belonging to that epoch. We retain the top $N_{\text{disc}} = 60$ features by F-score as the discriminative candidate set. However, since two features can both score highly on ANOVA while activating in entirely different patterns across epochs, we apply a second stage to enforce coherence. For each candiate feature we compute its epoch-mean activation profile $\mathbf{p}_i \in \mathbb{R}^K$, z scored across the K epoch categories. Features whose profiles correlate above $|\rho| > 0.5$ are connected in a graph, and connected components of size ≥ 2 are taken as clusters. If no clusters are found at this threshold we fall back to $|\rho| > 0.3$. We select the cluster containing the single most discriminative feature by ANOVA F-score as the candidate set for ablation, with each feature in the cluster ablated individually and independently.

For the cyclic concepts the graph clustering step is omitted and the top 60 features by ANOVA F-score are used directly as the candidate set, with integer cycle labels replacing epoch categories as the grouping variable.

4.6 Superposition Depth Scan

To examine how the temporal representation evolves across the network, we run a layer-wise ablation scan across all 12 layers of GPT2-Small. At each layer l we load the corresponding pretrained SAE, extract residual stream activations at `hook_resid_pre`, and apply the feature clustering procedure described in SECTION REF to identify the candidate feature set. We then establish a baseline temporal correlation $|r|_{\text{base}}$ by encoding the year prompts through the SAE, reconstructing via the decoder, and computing the abolsute Peasron correlation between PC1 and numeric year values as described in SECTION REF.

For each feature f_i in the candidate set, we zero its activation across all year prompts simultaneously, setting column i of the ($N_{\text{years}} \times 49{,}152$) feature activation matrix to zero in SAE space, and recompute the $|r|$ on the modified reconstruction. The drop is:

$$\Delta_i = |r|_{\text{base}} - |r|_{\text{abl},i}$$

A large Δ_i indicates that feature f_i is individually load-bearing for the temporal axis. A near-zero or negative Δ_i indicates negligible contribution or a suppressive feature whose removal improves temporal linearity. PCA is re-fit from scratch on each ablated reconstruction rather than projecting onto a fixed baseline direction, meaning Δ_i measures whether a temporal axis still exists after ablation, not merely whether the ablated reconstruction aligns with the original one. The test is therefore conservative — a feature only registers as load-bearing if the remaining features cannot reorganise to compensate.

5 Experiments and Results

5.1 BC/AD Token Contamination

TODO

5.2 Temporal Axis Construction

TODO

5.3 Superposition Transition

TODO

5.4 Load-Bearing Feature Identification

TODO

5.5 Circular Concepts: Days of Week and Months of Year

TODO

5.6 Days of the Month: Novel Geometric Analysis

TODO

6 Discussion

6.1 Linear vs Circular Temporal Encoding

TODO

6.2 The BC/AD Contamination Finding

TODO

6.3 Limitations

Need to think about clustering criteria for which cluster to take for ablating.
Current: Take the cluster that contains the single most discriminative features by ANOVA F-score
Could also try taking the largest cluster.
Need to include cluster sizes for ablation scans

7 Conclusion

TODO

References

J. Bloom. Gpt2 residual stream sparse autoencoders. `https://huggingface.co/jbloom/GPT2-Small-SAEs-Reformatted`, 2024. SAE release on Hugging Face, accessed 2024.

I. D. Coope. Circle fitting by linear and nonlinear least squares. *Journal of Optimization Theory and Applications*, 76(2):381–388, 1993.

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *International Conference on Learning Representations*, 2024. URL `https://arxiv.org/abs/2309.08600`.

A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL `https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf`.

A Additional Figures